Lecture 3

Convolutional Networks and Transfer Learning

Advanced Topics in Artificial Intelligence · UFABC

Adapted from MIT 15.773 (Farias, Ramakrishnan) — OCW

Part 1 — Computer Vision

How to represent images and what tasks to solve with DL.

Tensors and images

Tensor = N-dimensional array of numbers:

rank 0	rank 1	rank 2	rank 3	rank 4
escalar	vetor	matriz	cubo	rank-4
				
`42`	`[a, b, c, d]`	`imagem cinza`	`imagem RGB`	`batch RGB`
um número	lista de números	H × W	H × W × 3	N × H × W × 3

Grayscale image

Matrix $H \times W$ — each pixel: 0–255

Color image (RGB)

Tensor $H \times W \times 3$ — 3 channels (R, G, B)

Batch of images

Tensor $N \times H \times W \times 3$

Computer vision tasks

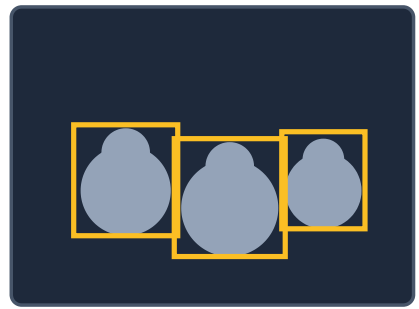
Classificação

qual é a classe?

Localização

classe + caixa

Detecção de objetos



várias classes/caixas

Segmentação semântica

classe por pixel

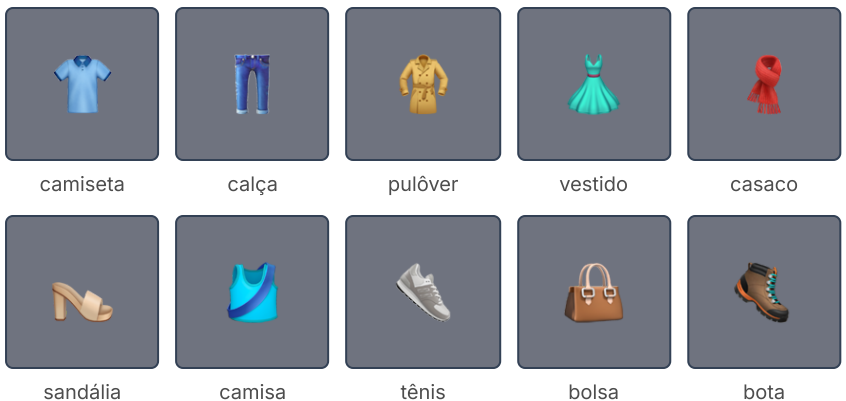
Classification
One class per image

Detection
Multiple bounding boxes + classes

Segmentation
Class per pixel

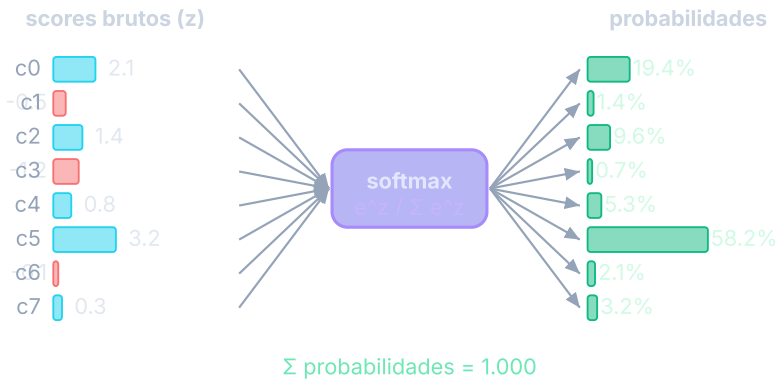
Multiclass classification — softmax

Fashion-MNIST: 70k images 28×28, 10 categories.



For N classes, the output layer uses **softmax**:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$



Outputs $\in (0, 1)$ summing to exactly 1.

Matching output to loss

Output variable	Output layer	Loss (Keras)
Real number	linear (1 neuron)	`mse`
Binary probability	sigmoid (1 neuron)	`binary_crossentropy`
Probability vector	**softmax** (N neurons)	`categorical_crossentropy`
Same, labels as integers	**softmax** (N neurons)	`sparse_categorical_crossentropy`

⚠ Using the wrong loss for the label type is one of the most common sources of bugs.

Baseline: dense network on Fashion-MNIST

Keras

```
inp   = keras.Input(shape=(28, 28))
x     = keras.layers.Flatten()(inp)
x     = keras.layers.Dense(128, 'relu')(x)
x     = keras.layers.Dropout(0.3)(x)
x     = keras.layers.Dense(64,  'relu')(x)
out   = keras.layers.Dense(10, 'softmax')(x)
model = keras.Model(inp, out)
model.compile(
  loss='sparse_categorical_crossentropy',
  optimizer='adam', metrics=['accuracy'])
```

PyTorch

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 64),  nn.ReLU(),
    nn.Linear(64, 10),
)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())
# CrossEntropyLoss already includes softmax!
```

Baseline of ~88% with dense layers. Can we do better?

Part 2 — Why dense layers are not enough

The problem of flattening images.

The problem with Flatten

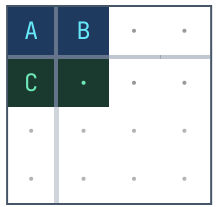
- Color image from a smartphone: **3024 × 3024 × 3 pixels**
- Flatten and connect to a layer of 100 neurons → **≈ 2.7 billion parameters**
- Computationally infeasible, requires enormous amounts of data, prone to overfitting

By flattening, we lose the **spatial structure** — neighboring pixels carry local information that the dense layer ignores.

If a pattern appears at different positions in the image, the dense network needs to **re-learn it** for each position. Convolutional filters **learn once and reuse** at any position.

Flatten: what is lost

4×4 image (original)



A–B: neighbors → ✓
A–C: neighbors ↓

→
Flatten

1×16 vector (after Flatten)

A–B: still neighbors ✓
A–C: now 4 positions apart ×

Parameter explosion
224×224×3 → 128 neurons = **19M params** in that layer alone

Loses invariance
Cat in the corner ≠ cat in the center — the network treats them as distinct inputs

Loses 2D adjacency
Vertical neighbors end up *width* positions apart in the vector

Part 3 — Convolutional Filters

How to detect visual patterns efficiently.

The convolutional filter

- A **filter** is a small matrix of numbers (e.g.: 3×3)
- Sliding over the image, it detects a type of visual pattern
- A **convolutional layer** has multiple filters; each one learns a different pattern

```
Vertical edge:    Horizontal edge:
 1  0 -1           1  1  1
 1  0 -1           0  0  0
 1  0 -1          -1 -1 -1
```

Analogy with dense neuron:

- Dense neuron → connected to **all** pixels
- Convolutional filter → connected to a **local window** and slides with the **same weights**

Benefits:

- **Far fewer parameters**
- Preserves **spatial adjacency**
- **Translation invariance** — detects the same pattern at any position

The convolution operation

Steps:

1. Position the filter over a window of the image
2. Multiply element-wise and sum → a scalar
3. Slide (stride) and repeat
4. Result: a **feature map**
5. Apply ReLU for non-linearity

```
3×3 window:   Filter (vertical edge):
1  2  3        1   0  -1
4  5  6   ×    1   0  -1   =  (1+4+7)-(3+6+9) = -6
7  8  9        1   0  -1
```

High value in the feature map = filter **detected** the pattern in that region.

By stacking layers:

- Layer 1 → edges and simple textures
- Layer 2 → corners, curves
- Layer 3+ → object parts, complete objects

Convolution step by step

6×6 input (excerpt) — 3×3 filter slides with stride 1:

```
┌─────┬─────┬─────┐ . . .
│  1  │  2  │  3  │ 0 1
│  4  │  5  │  6  │ 1 0 → position (0,0) → -6
│  7  │  8  │  9  │ 0 1
└─────┴─────┴─────┘ . . .


.  ┌─────┬─────┬─────┐ . .
.  │  2  │  3  │  0  │ → position (0,1) → 3
.  │  5  │  6  │  1  │
.  │  8  │  9  │  0  │
.  └─────┴─────┴─────┘ . .
```

```
Filter (vertical edge):
  1   0  -1
  1   0  -1
  1   0  -1
```

```
Resulting feature map (4×4):
-6   3  …
…    …
```

Output size: $(H - F + 1) \times (W - F + 1)$

6×6 input, 3×3 filter → **4×4** output

With **padding=1** → **6×6** output (same resolution)

What filters detect

Vertical edge

```
 1  0 -1
 1  0 -1
 1  0 -1
```

High response where intensity changes from left to right

Horizontal edge

```
 1  1  1
 0  0  0
-1 -1 -1
```

High response where intensity changes from top to bottom

Blur (smoothing)

```
1/9 1/9 1/9
1/9 1/9 1/9
1/9 1/9 1/9
```

Average of neighbors → smooths noise and details

In CNNs, **filter values are not hand-crafted** — they are **learned by backprop**. The network discovers on its own which patterns are useful for the task. Deep layers combine edges → corners → parts → objects.

Parameters of a convolutional layer

- **Filter size**: 3×3 or 5×5 (3×3 preferred)
- **Number of filters**: 32, 64, 128... per layer
- **Stride**: sliding step (1 or 2)
- **Padding**: add zeros at the borders to preserve resolution

Parameters of 32 filters 3×3 with 3 channels:

$$32 \times (3 \times 3 \times 3 + 1) = 896$$

Equivalent dense layer ($224 \times 224 \times 3 \to 32$):

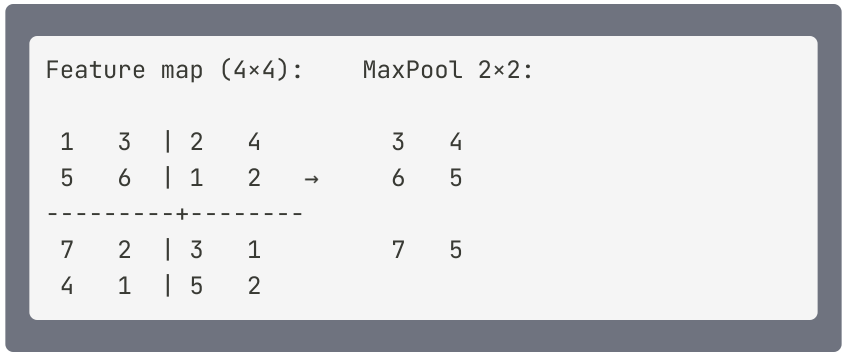
$$\approx 4.8 \text{ million parameters}$$

Part 4 — Pooling

Reducing dimensions without losing the essentials.

Max Pooling

- Divides the feature map into windows (e.g.: 2×2)
- Retains only the **maximum value** of each window
- Reduces spatial resolution by half
- Fewer parameters in subsequent layers



Intuition: if a feature exists **anywhere** in the window, max pooling preserves it. Provides robustness to small shifts.

Max Pooling vs Average Pooling

Max Pooling — preserves the *presence* of the feature

```
Feature map (4×4):
1   3 | 2   4
5   6 | 1   2
------+------
7   2 | 3   1
4   1 | 5   2

→ MaxPool 2×2:
6  4
7  5
```

Maximum of each window: "does this pattern **exist** here?"

Average Pooling — preserves the *average intensity*

```
Feature map (4×4):
1   3 | 2   4
5   6 | 1   2
------+------
7   2 | 3   1
4   1 | 5   2

→ AvgPool 2×2:
3.75  2.25
3.5   2.75
```

Average of each window: "what is the **typical** intensity of this region?"

MaxPool — standard in vision CNNs; detects feature presence

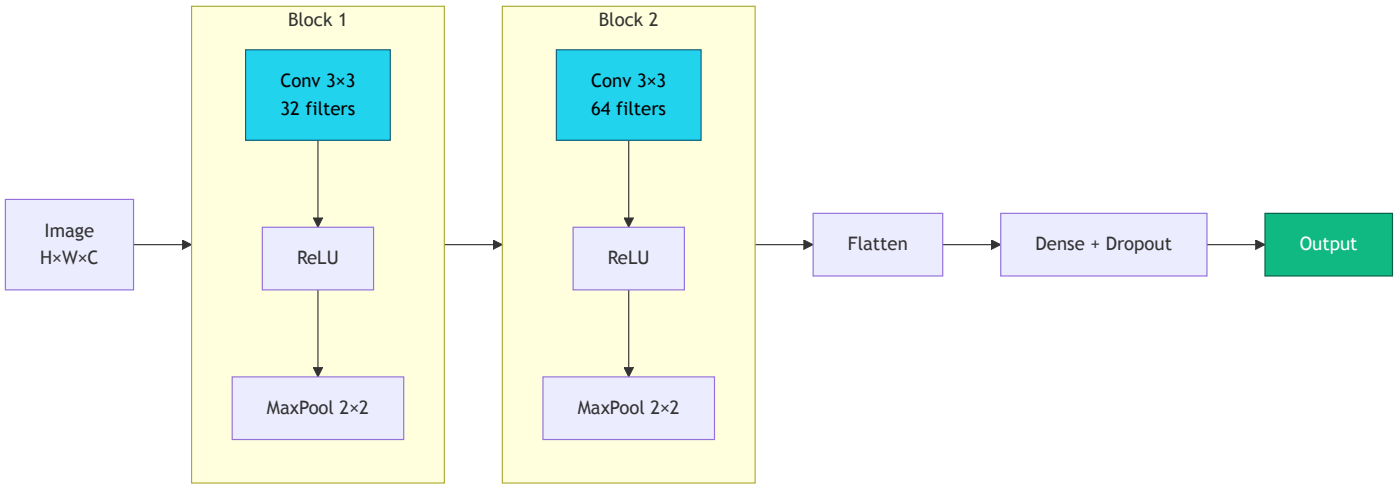
GlobalAvgPool — used before the classification head in modern networks (e.g.: ResNet); produces one vector per channel without

Part 5 — CNN Architecture

Combining convolutions, pooling and dense layers.

Convolutional blocks

A CNN is built from stacked **convolutional blocks** — each block has **greater depth** and **lower resolution** than the previous one — followed by dense layers:

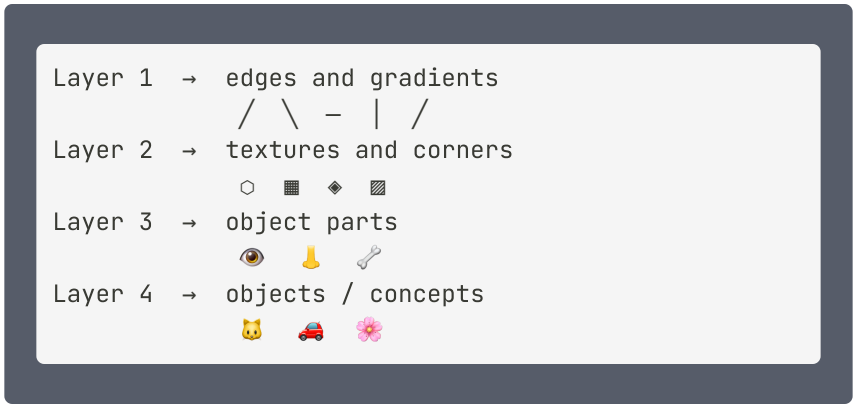


Stacking layers — hierarchy of patterns

Why stack convolutional blocks?

- Each layer sees a **local window** of the previous layer's output
- Deeper layers have a **larger receptive field** — they see more of the image
- The network learns a **hierarchy of representations**, from simple to complex

Empirical visualization (Zeiler & Fergus, 2014): filters of trained networks show a clear progression — early layers detect edges and gradients, middle layers detect textures and parts, deep layers detect complete objects.



Depth → increasing abstraction.

Transfer learning works because these representations **generalize** to new domains.

CNN for Fashion-MNIST

Keras

```
inp = keras.Input(shape=(28, 28, 1))
x = keras.layers.Conv2D(
      32, 3, activation='relu', padding='same')(inp)
x = keras.layers.MaxPooling2D()(x)
x = keras.layers.Conv2D(
      64, 3, activation='relu', padding='same')(x)
x = keras.layers.MaxPooling2D()(x)
x = keras.layers.Flatten()(x)
x = keras.layers.Dense(128, activation='relu')(x)
x = keras.layers.Dropout(0.3)(x)
out = keras.layers.Dense(10, activation='softmax')(x)
model = keras.Model(inp, out)
model.compile(
  loss='sparse_categorical_crossentropy',
  optimizer='adam', metrics=['accuracy'])
```

PyTorch

```
model = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Flatten(),
    nn.Linear(64 * 7 * 7, 128), nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 10),
)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters())
```

CNN achieves **~92%** on Fashion-MNIST, versus ~88% for the dense network.

Residual Connections (*Skip Connections*)

The problem with very deep networks

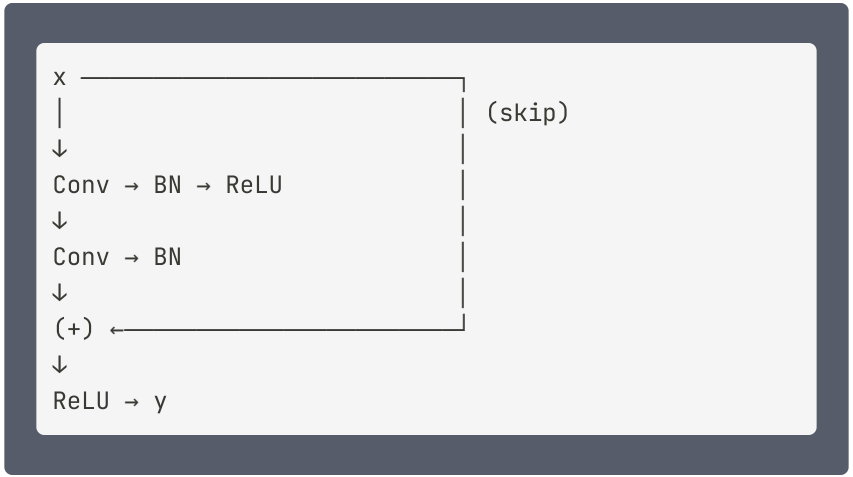
- Gradient dissipates layer by layer (*vanishing gradient*)
- Networks with >20 layers converged worse than shallow ones

Idea (He et al., ResNet 2015): allow the signal to flow directly by skipping layers.

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, W) + \mathbf{x}$$

Instead of learning $\mathbf{y}$, the block learns the **residual** $\mathcal{F} = \mathbf{y} - \mathbf{x}$.

If $\mathcal{F} \approx 0$, the block acts as an **identity** — never makes the previous result worse.



BN = *Batch Normalization*: normalizes the activations of each layer (mean 0, variance 1 per batch), speeding up convergence and stabilizing training.

$$y = \mathcal{F}(\mathbf{x}, W) + \mathbf{x}$$

Gradient flows through the skip without depending on $\mathcal{F}$ → ResNet-152 trains successfully.

Part 6 — Transfer Learning

Reusing what was learned from millions of images.

Two trends that enabled transfer learning

Trend 1 — Specialized architectures:

Data type	Architecture
Any	Residual connections (ResNet)
Images	Convolutional layers
Sequences (text, audio, genes)	Transformers

Trend 2 — Publicly available pre-trained models:
Using these architectures, researchers trained high-performance networks on large real-world datasets. Countless models are publicly available.

The core idea of transfer learning

- **ImageNet**: millions of images from 1000 everyday categories
- A network trained on ImageNet develops a **hierarchical** representation of visual objects:
  - Early layers → edges and textures
  - Middle layers → object parts
  - Final layers → complete objects

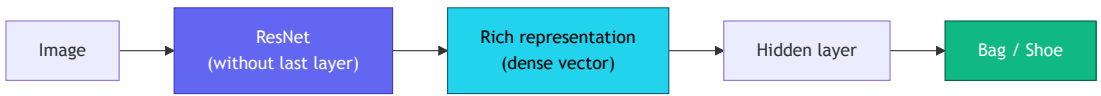
Problem: we only have ~100 images of bags and shoes. A CNN trained from scratch will not generalize.

Solution: take a ResNet trained on ImageNet and **reuse** what it already learned about everyday images.

Transfer learning = **customizing** a pre-trained network for a new problem.

"Headless" ResNet as a feature extractor

1. The original ResNet classifies 1000 categories — the last layer is not useful to us
2. We remove the last layer (**"headless" ResNet**)
3. We pass our images → the output is a rich, dense representation
4. We train a small network on top of that representation



With only ~100 images, this approach already yields high accuracy.

Feature extraction vs. Fine-tuning

Feature extraction

- Pre-trained base weights remain **frozen**
- Only the new head is trained
- Faster, works with little data
- Recommended when the target dataset is small and similar to ImageNet

Fine-tuning

- Connect base + head and train **everything end-to-end**
- Must start from pre-trained weights (not from scratch)
- Uses a smaller learning rate to avoid "forgetting"
- Better when there is sufficient data

Catastrophic Forgetting

The problem

- During fine-tuning, gradients from the new task **overwrite** weights important for the original task
- The network "forgets" what it learned during pre-training — loses the generic feature representation
- The higher the learning rate and the more layers unfrozen, the more severe the forgetting

Symptom: accuracy on the target dataset rises quickly, but the model loses generalization capacity — especially with limited data.

Mitigation strategies

Low learning rate
Fine-tune with LR 10–100× smaller than original pre-training (e.g. 1e-4 instead of 1e-2)

Gradual unfreezing
Unfreeze layers top-to-bottom, one at a time, over the course of training

Discriminative learning rates
Layers near the input get a smaller LR; layers in the head get a larger LR

Feature extraction first
Train only the head until convergence; then unfreeze for fine-tuning

Transfer learning in code

Keras

```
base = keras.applications.ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3),
)
base.trainable = False   # freeze

inp = keras.Input(shape=(224, 224, 3))
x   = base(inp, training=False)
x   = keras.layers.GlobalAveragePooling2D()(x)
x   = keras.layers.Dense(64, activation='relu')(x)
out = keras.layers.Dense(2,  activation='softmax')(x)

model = keras.Model(inp, out)
model.compile(
  loss='sparse_categorical_crossentropy',
  optimizer='adam', metrics=['accuracy'])
```

PyTorch

```
import torchvision.models as models

base = models.resnet50(weights='IMAGENET1K_V2')

# Freeze all weights
for p in base.parameters():
    p.requires_grad = False

# Replace the classification head
n_feat = base.fc.in_features
base.fc = nn.Sequential(
    nn.Linear(n_feat, 64),
    nn.ReLU(),
    nn.Linear(64, 2),
)

optimizer = optim.Adam(base.fc.parameters())
criterion = nn.CrossEntropyLoss()
```

Where to find pre-trained models

🔵

TensorFlow Hub

tensorflow.org/hub

Ready-to-use Keras models

🟠

PyTorch Hub

pytorch.org/hub

ResNet, EfficientNet, BERT...

🤗

Hugging Face Hub

huggingface.co/models

+500k models: vision, NLP, audio

Part 7 — Self-Supervised Learning

Why pre-train without labels?

The problem

- Labeling data is **expensive and slow**
- Annotated datasets: ImageNet (1.2 M), CIFAR-10 (60 K)
- Unlabeled data: **the entire internet**

The idea

- Train the backbone on an **auxiliary task** created from the data itself
- No human; the label is generated automatically
- The learned backbone is reused (feature extraction / fine-tuning) with few labeled examples

Self-Supervised Learning (SSL) = supervision comes from the data itself, not from human annotators.

The auxiliary task is called a **pretext task**.

Pretext Tasks

A pretext task automatically creates **(input, label)** pairs, forcing the backbone to learn useful representations.

🔄 Rotation Prediction

Rotates the image by 0°/90°/180°/270°. Predicts which rotation was applied.
Gidaris et al., 2018 — RotNet

🧩 Shuffled Patches

Divides into 9 patches and shuffles them. Predicts the original permutation.
Noroozi & Favaro, 2016 — Jigsaw

🎭 Masked Reconstruction

Masks ~75% of patches. Reconstructs the masked pixels.
He et al., 2022 — MAE

🔀 Contrastive Learning

Maximizes similarity between views of the same image; minimizes between different ones.
Chen et al., 2020 — SimCLR

🎨 Colorization

Receives a grayscale image. Predicts the colors (channels a and b of Lab color space).
Zhang et al., 2016

📍 Context Prediction

Extracts two random patches. Predicts the relative position between them (8 directions).
Doersch et al., 2015

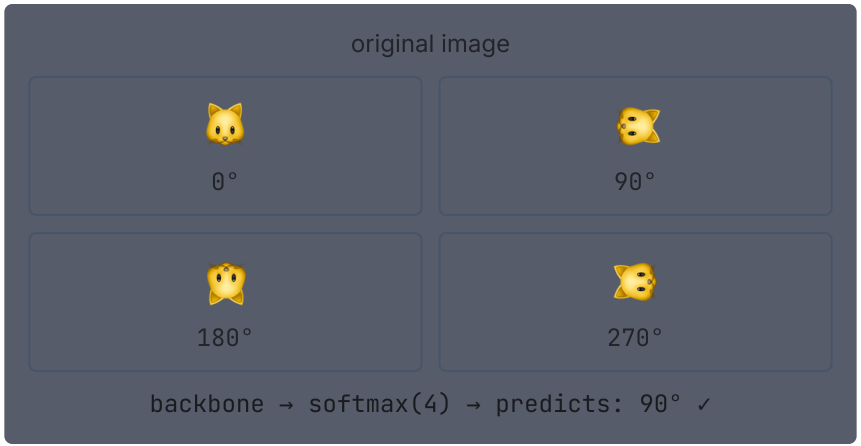
🔊 Denoising Autoencoder

Corrupts the input with noise. Reconstructs the original clean image.
Vincent et al., 2008 — DAE

Rotation Prediction (RotNet)

How it works

1. Takes any image (without label)
2. Applies one of 4 rotations: 0°, 90°, 180°, 270°
3. Backbone extracts features; classification head predicts which rotation (4 classes)
4. Loss: standard Cross-Entropy

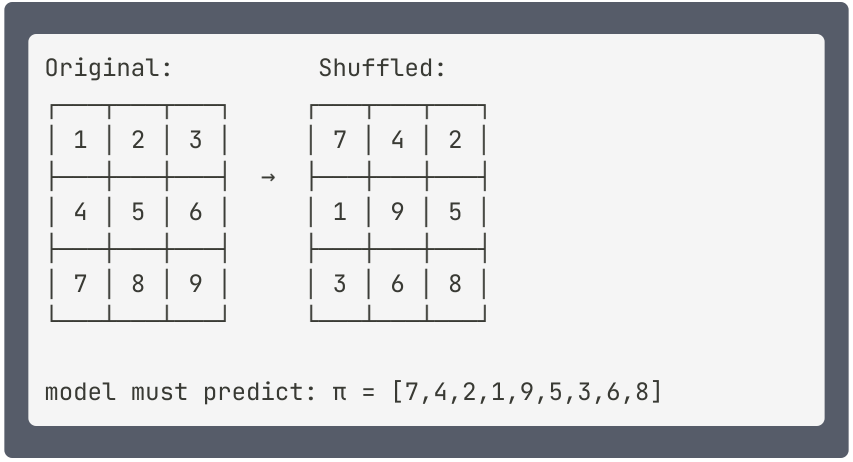


Why does it work? To get the rotation right, the model must understand *what is in the image* — the natural orientation of objects. This forces the backbone to learn semantic features.

Shuffled Patches (Jigsaw)

Procedure

1. Divides the image into a 3×3 grid (9 patches)
2. Shuffles the patches with a random permutation π
3. Backbone processes each patch; head predicts π from a pre-defined set
4. Backbone learns spatial relationships and semantic context



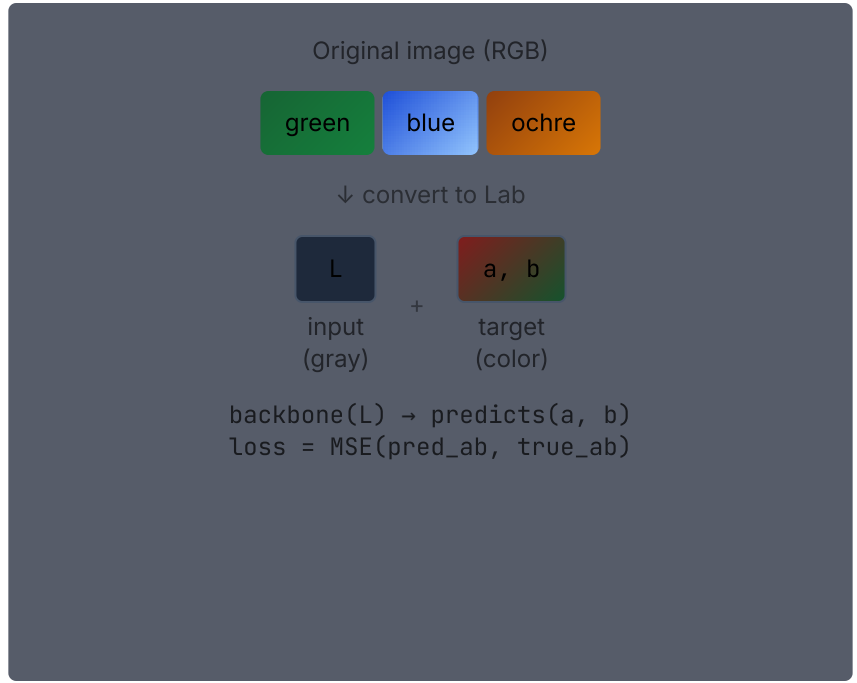
Intuition: To reassemble the image, the model must understand **what belongs together** — a key concept for object recognition.

Colorization

Procedure

1. Converts the image to the **Lab** color space
   - Channel **L** = luminosity (grayscale)
   - Channels **a, b** = chromaticity (color)
2. Provides only the **L** channel to the backbone
3. Backbone predicts the **a** and **b** channels
4. Loss: mean squared error on the color channels

Why does it work? To colorize correctly, the model must identify *what* each object is — grass is green, sky is blue, skin has specific tones. This forces semantic learning.



Advantage: label comes from the original image — no human annotation needed. Any color photo can serve as training data.

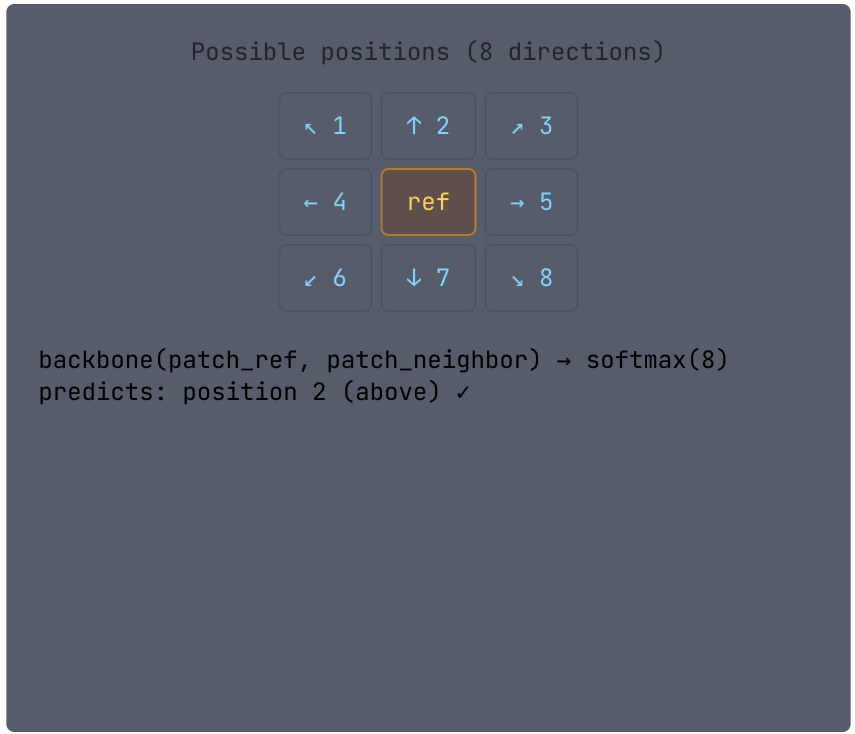
Limitation: color ambiguity (cars can be any color). Models learn to predict "safe" average colors; recent versions use probabilistic distributions.

Context Prediction (*Relative Patch Position*)

Procedure (Doersch et al., 2015)

1. Extracts the **central** patch of the image
2. Extracts a **neighboring** patch from one of the 8 surrounding positions
3. Backbone processes both patches
4. Head predicts which of the **8 directions** the neighbor occupies
5. Loss: Cross-Entropy (8 classes)

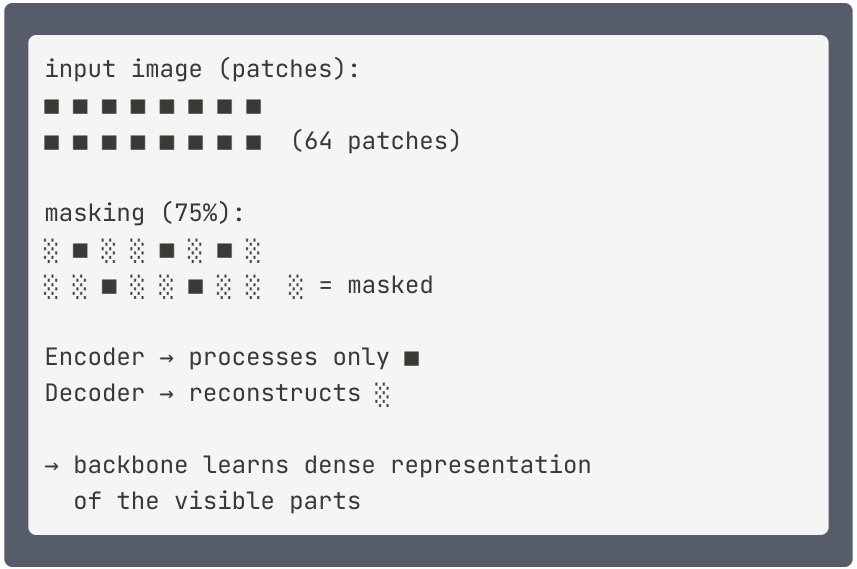
Why does it work? To get the relative position right, the model must understand object parts and their spatial relations — e.g., eyes are above the nose, wheels are below the car.



Masked Autoencoders (MAE)

Idea (He et al., 2022)

- Divides the image into 16×16 patches
- Masks ~75% of patches randomly
- **Encoder** (ViT) processes only visible patches
- Lightweight **decoder** reconstructs the masked pixels
- Loss: MSE on masked pixels



Why 75%? Small masks allow trivial interpolation. 75% forces global scene understanding.

Result: MAE pre-trains ViT-L on ImageNet in ~31 h (8 A100 GPUs) and achieves SOTA in classification with fine-tuning.

Contrastive Learning (SimCLR)

Principle

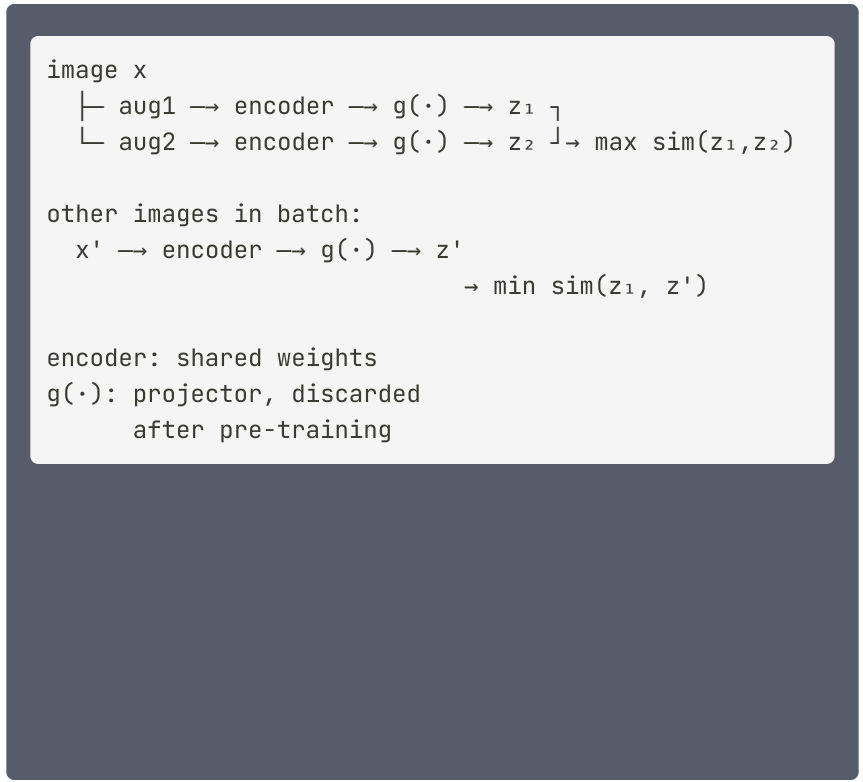
- Creates 2 **augmented views** of the same image (crop, flip, color jitter...)
- Maximizes similarity between views of the **same** image
- Minimizes similarity with **other images** in the batch

NT-Xent Loss (InfoNCE):

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

τ = temperature; z = projected embedding

Large batch → more negatives → better signal. SimCLR uses batch size 4096.



Denoising Autoencoder (DAE)

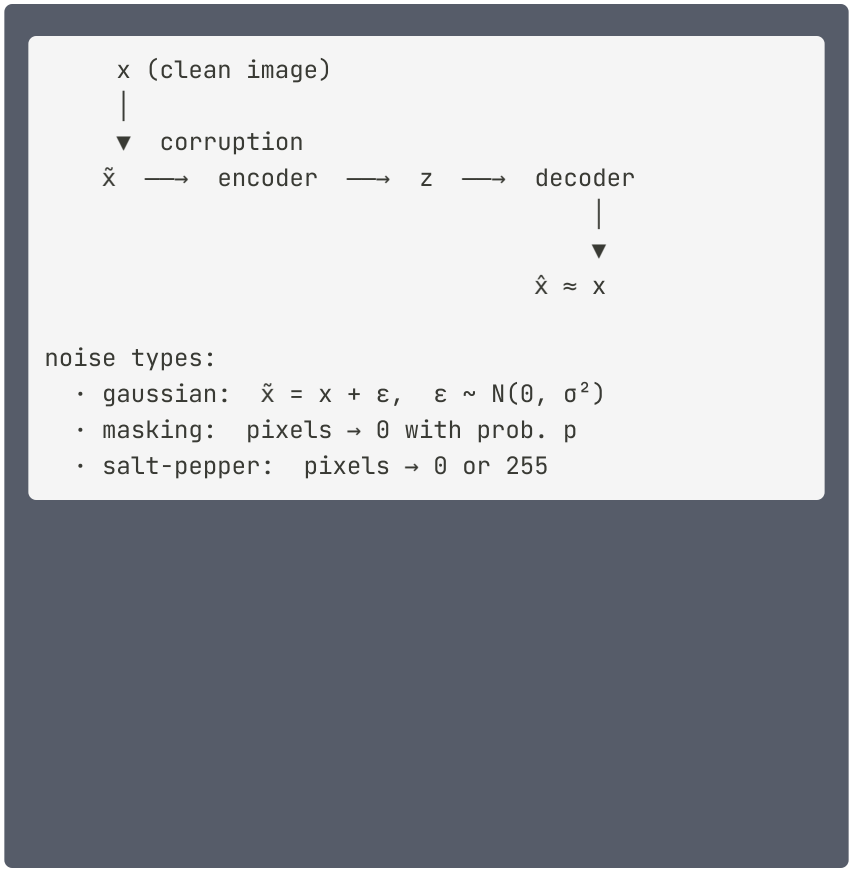
Core idea (Vincent et al., 2008)

- Corrupts the input: Gaussian noise, *salt-and-pepper*, region masking
- The model learns to map the corrupted version $\tilde{x}$ back to the original x
- The encoder is forced to capture semantic structure — it cannot memorize noise

Reconstruction loss:

$$\mathcal{L} = \|x - \text{dec}(\text{enc}(\tilde{x}))\|^2$$

Difference from MAE: DAE uses continuous noise on pixels; MAE masks entire patches. MAE scales better for ViTs.



SSL Methods Comparison

Method	Signal type	Architecture	Strengths
RotNet	Classification (4 classes)	CNN	Simple; good for quick pretraining
Jigsaw	Permutation classification	CNN	Learns spatial relationships
Colorization	Regression (Lab colors)	CNN encoder-decoder	Learns semantics of natural objects
Context Pred.	Classification (8 directions)	CNN	Learns spatial scene layout
DAE	Reconstruction (noise → clean)	CNN encoder-decoder	Robust to corruptions; historical pioneer
MAE	Pixel reconstruction	ViT + decoder	SOTA on ViTs; efficient
SimCLR	Contrastive (similarity)	CNN/ViT	Label-free; very flexible
DINO	Self-distillation	ViT	Emergent segmentation features

Generative methods (MAE, Jigsaw): the model reconstructs/reorders data; learns detailed geometry and texture.

Contrastive methods (SimCLR, DINO): the model learns invariances; learns high-level semantics.

Summary

Tensor H×W×3 — representation of color images; batch: N×H×W×3

Softmax + Cross-Entropy — multiclass output; correct loss for N categories

Flatten + Dense — simple baseline, but explodes in parameters and loses spatial structure

Convolutional filter — local window with shared weights; learns once, applies across the entire image

Max Pooling — downsampling that preserves feature presence; robustness to shifts

CNN — Conv+ReLU+Pool blocks; depth grows, resolution shrinks

Feature extraction — frozen base + new head; works with little data

Fine-tuning — trains everything end-to-end starting from pre-trained weights

Pretext task — automatically generated label from the data; forces the backbone to learn useful representations

SSL — RotNet, Jigsaw, MAE, SimCLR; pre-training without human annotations

Next lecture

- **Sequences and language** — how to represent text for neural networks
- **Embeddings** — dense vectors for words and tokens
- **RNNs and LSTMs** — networks that process sequences step by step
- **Attention and Transformers** — the mechanism that dominates modern NLP
- **BERT and GPT** — pre-training and fine-tuning for language tasks

Thank you! Questions?